



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

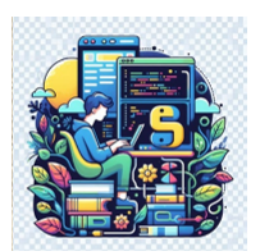
PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Encapsulation

Lekha A

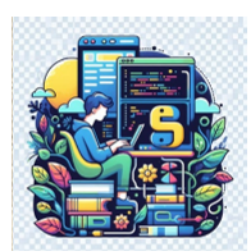
Computer Applications



PROGRAMMING WITH PYTHON

Encapsulation

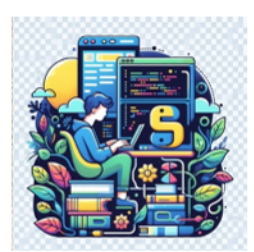
- Mechanism that binds together code and data it manipulates and keeps both safe from outside interference and misuse.
- Encapsulation allows programmers better control of how data flows in their programs, and it protects that data.
- It also makes objects into more self-sufficient and independently functioning pieces.



PROGRAMMING WITH PYTHON

Encapsulation

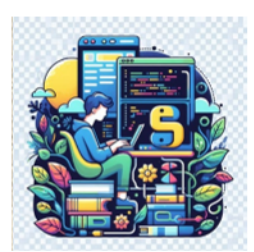
- Encapsulation is a mechanism of wrapping the data (variables) and code acting on the data (methods) together as a single unit.
- In encapsulation, the variables of a class will be hidden from other classes, and can be accessed only through the methods of their current class.



PROGRAMMING WITH PYTHON

Encapsulation

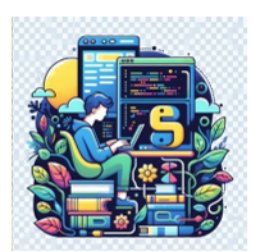
- Example
 - A company is selling courses to students, engineers and professionals.
 - The different sections of this company include, operations, finance, accounts, sales, etc.
 - Now, if an employee from the accounts section needs the records of sales in 2022, then he/ she cannot directly access it.



PROGRAMMING WITH PYTHON

Encapsulation

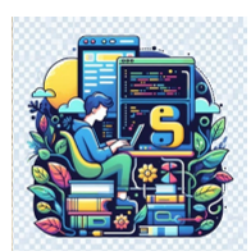
- To access, the employee of the account sections needs to get permission from the sales section team member.
 - The sales data is hidden from other departments.
- In the same way, the financials of the company is accessible to only the finance department and is hidden from other sections.
- The accounts, sales, finance, operations, marketing, etc. data is hidden from other sections



PROGRAMMING WITH PYTHON

Encapsulation

- Class is an example of encapsulation.
 - A class combines data and methods into a single unit.
- In the example the custom function `demofunc()` displays the records of students where the public data member can be accessed.
 - Using the objects `st1`, `st2`, `st3`, `st4`, the public methods of the class `demofunc()` are accessed.

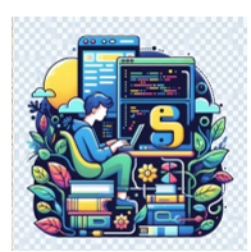


PROGRAMMING WITH PYTHON

Encapsulation

```
• class Students:
    def __init__(self, name, rank, points):
        self.name = name
        self.rank = rank
        self.points = points

    # custom function
    def demofunc(self):
        print("I am "+self.name)
        print("I got Rank ",+self.rank)
```

PROGRAMMING WITH PYTHON

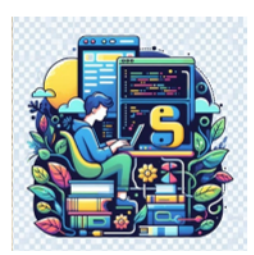
Encapsulation

- # create 4 objects*

```
st1 = Students("Krish", 1, 100)
st2 = Students("Ram", 2, 90)
st3 = Students("Rudra", 3, 76)
st4 = Students("Avi", 4, 60)
```

call the functions using the objects created above

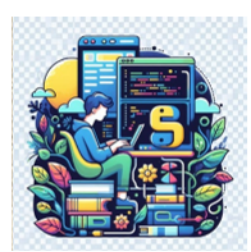
```
st1.demofunc()
st2.demofunc()
st3.demofunc()
st4.demofunc()
```



PROGRAMMING WITH PYTHON

Access Modifiers

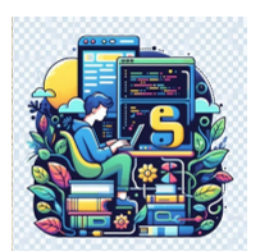
- Python has different levels of restriction that control how data can be accessed and from where.
- Variables and methods can be public, private, or protected.
- Those designations are made by the number of underscores before the variable or method.



PROGRAMMING WITH PYTHON

Access Modifiers

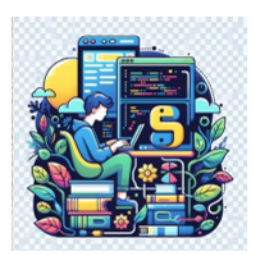
- Public
 - Public variables and methods can be freely modified and run from anywhere, either inside or outside of the class.
 - To create a public variable or method, don't use any underscores.



PROGRAMMING WITH PYTHON

Access Modifiers

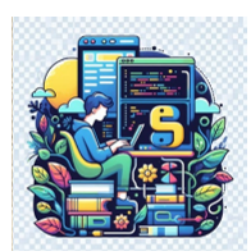
- Private
 - The private designation only allows a variable or method to be accessed from within its own class or object.
 - The value of a private variable cannot be modified from outside of a class.
 - Private variables and methods are preceded by two underscores.



PROGRAMMING WITH PYTHON

Access Modifiers

- Protected
 - Protected variables and methods are very similar to private ones.
 - A variable that is protected can only be accessed by its own class and any classes derived from it.
 - Protected variables begin with a single underscore.



PROGRAMMING WITH PYTHON

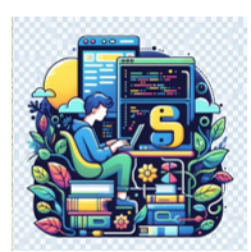
Access Modifiers

```
class Employee:  
    def __init__(self, name, project, salary):  
        self.name= name  
  
        self._project=project  
  
        self.__salary=salary
```

Public member (Accessible within or outside of a class)

Protected member (Accessible within the class and its subclasses)

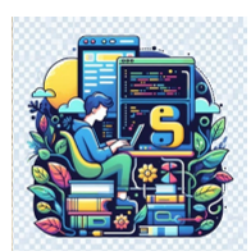
Private member (Accessible only within the class)



PROGRAMMING WITH PYTHON

Access Modifiers

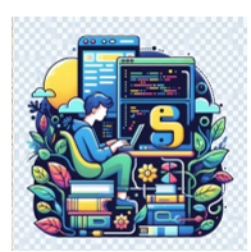
- Encapsulation is the process of using private variables within classes to prevent unintentional or potentially malicious modification of data.
- By containing and protecting variables within a class, it allows the class and the objects that it creates to function as independent, self contained, parts functioning within the machine of the program itself.



PROGRAMMING WITH PYTHON

Access Modifiers

- To encapsulate data in Python, the following steps have to be done:
 - Define a class and declare the data members as private.
 - Provide public methods to access and modify the data members.
 - Implement the public methods in a way that protects the data members from being modified accidentally or maliciously.

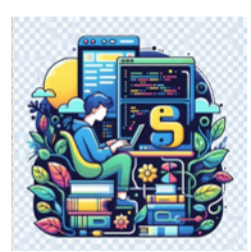


PROGRAMMING WITH PYTHON

Access Modifiers - Public

```
class Employee:
    # constructor
    def __init__(self, name, salary):
        # public data members
        self.name = name
        self.salary = salary

    # public instance methods
    def show(self):
        # accessing public data member
        print("Name: ", self.name, 'Salary:', self.salary)
```



PROGRAMMING WITH PYTHON

Access Modifiers - Public

- # creating object of a class*

```
emp = Employee('Avi', 100000)
```

accessing public data members

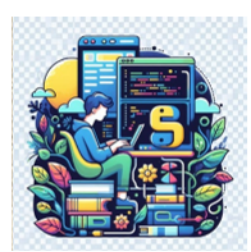
```
print("Name: ", emp.name, 'Salary:', emp.salary)
```

```
Name:  Avi Salary: 100000
```

calling public method of the class

```
emp.show()
```

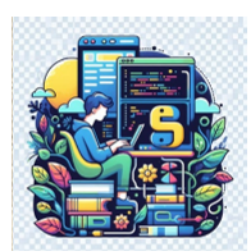
```
Name:  Avi Salary: 100000
```



PROGRAMMING WITH PYTHON

Access Modifiers - Private

- ```
class Employee:
 # constructor
 def __init__(self, name, salary):
 # public data member
 self.name = name
 # private member
 self.__salary = salary
```



# PROGRAMMING WITH PYTHON

## Access Modifiers - Private

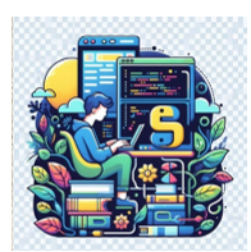
```
creating object of a class
emp = Employee('Rudra', 90000)
```

```
accessing private data members
print('Salary:', emp.__salary)
```

```

AttributeError Traceback (most recent call last)
Cell In[10], line 2
 1 # accessing private data members
----> 2 print('Salary:', emp.__salary)

AttributeError: 'Employee' object has no attribute '__salary'
```

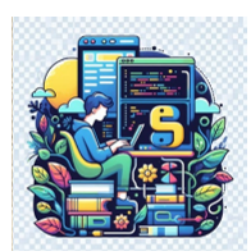


# PROGRAMMING WITH PYTHON

## Access Modifiers - Private

---

- To access private members from outside of a class using the following two approaches
  - Create public method to access private members
  - Use name mangling



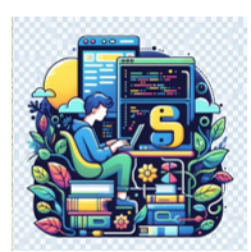
# PROGRAMMING WITH PYTHON

## Access Modifiers - Private

- Public method to access private members

```
class Employee:
 # constructor
 def __init__(self, name, salary):
 # public data member
 self.name = name
 # private member
 self.__salary = salary

 # public instance methods
 def show(self):
 # private members are accessible from a class
 print("Name: ", self.name, 'Salary:', self.__salary)
```



# PROGRAMMING WITH PYTHON

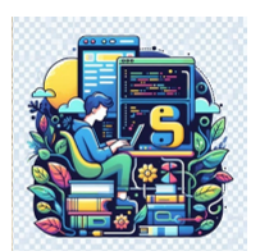
## Access Modifiers - Private

- Public method to access private members

- ```
# creating object of a class  
emp = Employee('Avi', 100000)
```

```
# calling public method of the class  
emp.show()
```

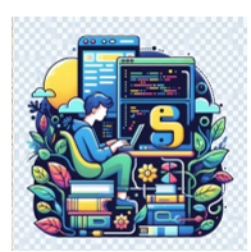
```
Name:  Avi Salary: 100000
```



PROGRAMMING WITH PYTHON

Access Modifiers - Private

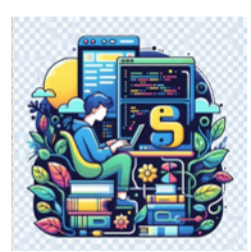
- Name Mangling to access private members
 - Private and protected variables can be accessed from outside of a class through name mangling.
 - The name mangling is created on an identifier by adding two leading underscores and one trailing underscore.



PROGRAMMING WITH PYTHON

Access Modifiers - Private

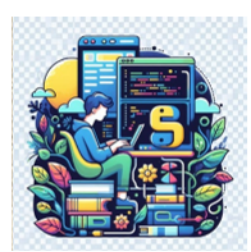
- Name Mangling to access private members
 - Name mangling in Python is the process of modifying the name of an identifier to make it unique.
 - This is done by adding two leading underscores to the name of the identifier.



PROGRAMMING WITH PYTHON

Access Modifiers - Private

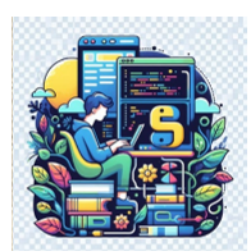
- Name Mangling to access private members
 - Name mangling is used to avoid name collisions between class attributes and instance attributes.
 - `_classname__dataMember`, where `classname` is the current class, and `data member` is the private variable name.



PROGRAMMING WITH PYTHON

Access Modifiers - Private

- ```
class Employee:
 # constructor
 def __init__(self, name, salary):
 # public data member
 self.name = name
 # private member
 self.__salary = salary
```



# PROGRAMMING WITH PYTHON

## Access Modifiers - Private

- # creating object of a class*

```
emp = Employee('Rudra', 90000)
```

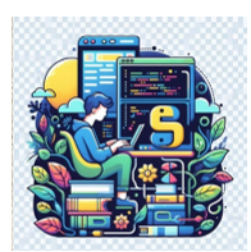
```
print('Name:', emp.name)
```

Name: Rudra

- # direct access to private member using name mangling*

```
print('Salary:', emp._Employee__salary)
```

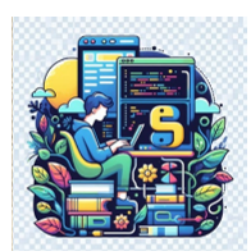
Salary: 90000



# PROGRAMMING WITH PYTHON

## Access Modifiers - Protected

- ```
class Employee:  
    # constructor  
    def __init__(self, name, salary):  
        # public data member  
        self.name = name  
        # private member  
        self.__salary = salary
```



PROGRAMMING WITH PYTHON

Access Modifiers - Protected

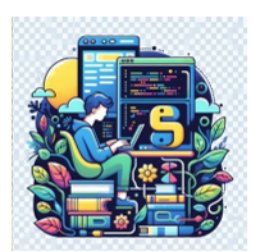
- ```
c = Employee("Avi")
c.show()
```

Employee name : Avi

Working on project : NLP

```
Direct access protected data member
print('Project:', c._project)
```

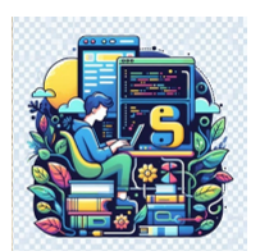
Project: NLP



# PROGRAMMING WITH PYTHON

## Access Modifiers

| Class Member<br>Access Specifier | Access from<br>own class | Access from<br>derived class | Access from<br>object |
|----------------------------------|--------------------------|------------------------------|-----------------------|
| Private                          | Yes                      | No                           | No                    |
| Protected                        | Yes                      | Yes                          | No                    |
| Public                           | Yes                      | Yes                          | Yes                   |



# PROGRAMMING WITH PYTHON

## Recap

- Encapsulation
  - Public Method to access private members
- Example
  - Name Mangling to access private members
- Access Specifiers
  - Public
  - Private
  - Protected





**THANK YOU**

---

**Lekha A**  
Computer Applications